

Taller práctico de Git

Crear una simulación de “página web” para una cafetería llamada Café Aurora.

No usaremos código complejo.

El objetivo es aprender Git paso a paso como en un entorno profesional.

Usaremos:

- ramas
- commits
- merge
- stash
- tags
- remotos
- conflictos
- restauración
- historial
- rebase
- etc.

Caso práctico

Tiene esta carpeta: `cafe-aurora/`

Dentro habrá archivos simples:

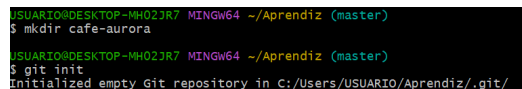
```
index.html  
menu.txt  
contacto.txt  
README.md
```

PARTE 1 Crear repositorio

1) git init

Inicializa Git.

```
git init
```



```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)  
$ mkdir cafe-aurora  
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)  
$ git init  
Initialized empty Git repository in C:/Users/USUARIO/Aprendiz/.git/
```

Resultado:git

```
Initialized empty Git repository
```

2) git status

Ver estado actual.

```
git status
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)
$ git status
On branch master

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

3) git config

Configurar usuario.

```
git config --global user.name "Juan"
git config --global user.email "juan@email.com"
```

Ver configuración:

```
git config --list
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)
$ git config --global user.name "Daniela Londoño"
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)
$ git config --global user.email "daniela.londono999@gmail.com"
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)
$ git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=openssl
core.autocrlf=true
core.fscache=true
core.sylmlinks=false
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=master
user.name=Daniela Londoño
user.email=daniela.londono999@gmail.com
core.repositoryformatversion=0
core.filemode=false
core.bare=false
core.logallrefupdates=true
core.sylmlinks=false
core.ignorecase=true
```

PARTE 2 Primer commit

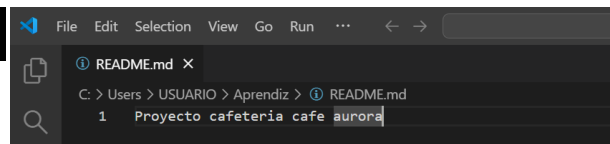
Crea:

README.md

Contenido:

Proyecto cafetería Cafe Aurora

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)
$ touch README.md
```



4) git add

```
git add README.md
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)
$ git add README.md
```

5) git status

```
git status
```

Verás:

```
Changes to be committed
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   README.md
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)
```

6) git commit

```
git commit -m "Primer commit"
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)
$ git commit -m "Primer commit"
[master (root-commit) fd94e4b] Primer commit
1 file changed, 1 insertion(+)
create mode 100644 README.md
```

7) git log

```
git log
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)
$ git log
commit fd94e4bacf7a1cc35a7e7ed8fd9de38c1ab1a0e5 (HEAD -> master)
Author: Daniela Londoño <daniela.londono999@gmail.com>
Date:   Wed May 27 15:27:27 2026 -0500

    Primer commit
```

8) git log --oneline

```
git log --oneline
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)
$ git log --oneline
fd94e4b (HEAD -> master) Primer commit
```

PARTE 3 Trabajar con archivos

Crea:

menu.txt

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)
$ touch menu.txt
```

9) git add .

git add .

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)
$ git add .
```

10) git diff

Modifica menu.txt.

Luego:

git diff

Muestra cambios no staged.

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)
$ git diff
```

11) git commit

git commit -m "Agrega menú inicial"

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)
$ git commit -m "Agrega menú inicial"
[master b12296a] Agrega menú inicial
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 menu.txt
```

PARTE 4 Branches

12) git branch

git branch

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)
$ git branch
* master
```

13) git branch feature/contacto

git branch feature/contacto

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)
$ git branch feature/contacto
```

14) git checkout

```
git checkout feature/contacto
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)
$ git checkout feature/contacto
Switched to branch 'feature/contacto'
```

15) git switch

Volver a main:

```
git switch main
```

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (feature/contacto)
$ git switch main
fatal: invalid reference: main

USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (feature/contacto)
$ git switch main
fatal: invalid reference: main

USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (feature/contacto)
$ git branch
* feature/contacto
  master

USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (feature/contacto)
$ git branch
* feature/contacto
  master

USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (feature/contacto)
$ git switch master
Switched to branch 'master'

USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)
$ git switch main
fatal: invalid reference: main

USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)
$ git branch -a
  feature/contacto
* master
```

16) git checkout -b

Crear y cambiar:

```
git checkout -b feature/redes-sociales
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)
$ git checkout -b feature/redes-sociales
Switched to a new branch 'feature/redes-sociales'
```

PARTE 5 Merge

Agrega:

```
instagram: @cafeaurora
```

Commit:

```
git add .
git commit -m "Agrega redes sociales"
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (feature/redes-sociales)
$ git add .
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (feature/redes-sociales)
$ git commit -m "Agrega redes sociales"
On branch feature/redes-sociales
nothing to commit, working tree clean
```

17) git merge

Volver a main:

```
git switch main
```

Fusionar:

```
git merge feature/redes-sociales
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (feature/redes-sociales)
$ git switch master
Switched to branch 'master'
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)
$ git merge feature/redes-sociales
Already up to date.
```

PARTE 6 Eliminar y mover archivos

18) git rm

```
git rm contacto.txt
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)
$ touch contacto.txt
```

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)
$ git add contacto.txt
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)
$ git rm contacto.txt
error: the following file has changes staged in the index:
    contacto.txt
(use --cached to keep the file, or -f to force removal)
```

19) git mv

```
git mv menu.txt menu-cafeteria.txt
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)
$ git mv menu.txt menu-cafeteria.txt
```

20) git commit

```
git commit -m "Reorganiza archivos"
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)
$ git commit -m "Reorganiza archivos"
[master 2314044] Reorganiza archivos
2 files changed, 0 insertions(+), 0 deletions(-)
rename menu.txt => contacto.txt (100%)
create mode 100644 menu-cafeteria.txt
```

PARTE 7 Restaurar cambios

Modifica README.

21) git restore

```
git restore README.md
```

Descarta cambios.

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)  
$ git restore README.md
```

22) git restore --staged

```
git restore --staged README.md
```

Quita del staging.

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)  
$ git restore --staged README.md
```

PARTE 8 Trabajar con remoto (GitHub)

23) git remote add

```
git remote add origin https://github.com/juan/cafe-aurora.git  
https://github.com/Danielalon99/cafe-aurora
```

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)  
$ git remote add origin https://github.com/Danielalon99/cafe-aurora
```

24) git remote -v

```
git remote -v  
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)  
$ git remote -v  
origin https://github.com/Danielalon99/cafe-aurora (fetch)  
origin https://github.com/Danielalon99/cafe-aurora (push)
```

25) git push

```
git push -u origin main
```

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)  
$ git push -u origin master  
fatal: unable to access 'https://github.com/Danielalon99/cafe-aurora/': Could not resolve host: github
```

26) git pull

```
git pull
```

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)
$ git pull
info: please complete authentication in your browser...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 874 bytes | 14.00 KiB/s, done.
From https://github.com/Danielalon99/cafe-aurora
* [new branch]      main      -> origin/main
There is no tracking information for the current branch.
Please specify which branch you want to merge with.
See git-pull(1) for details.

    git pull <remote> <branch>

If you wish to set tracking information for this branch you can do so with:

    git branch --set-upstream-to=origin/<branch> master
```

27) git fetch

```
git fetch
```

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)
$ git fetch
```

28) git clone

Otro desarrollador haría:

```
git clone https://github.com/juan/cafe-aurora.git
```

```
git clone https://github.com/Danielalon99/cafe-aurora
```

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)
$ git fetch

USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)
$ git clone https://github.com/Danielalon99/cafe-aurora
Cloning into 'cafe-aurora'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
```

PARTE 9 Stash

Estás trabajando y necesitas cambiar de rama rápido.

29) git stash

```
git stash
```

Guarda temporalmente cambios.

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)
$ git stash
No local changes to save
```


30) git stash list

```
git stash list
```

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)  
$ git stash list
```

31) git stash pop

```
git stash pop
```

Recupera cambios.

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)  
$ git stash pop  
No stash entries found.
```

PARTE 10 Historial y detalles

32) git show

```
git show
```

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)  
$ git show  
commit 2314044f844d51fa831fae376490a29f79cb4f7a (HEAD -> master)  
Author: Daniela Londoño <daniela.londono999@gmail.com>  
Date: Wed May 27 16:05:29 2026 -0500  
  
    Reorganiza archivos  
  
diff --git a/menu.txt b/contacto.txt  
similarity index 100%  
rename from menu.txt  
rename to contacto.txt  
diff --git a/menu-cafeteria.txt b/menu-cafeteria.txt  
new file mode 100644  
index 0000000..e69de29
```

33) git reflog

```
git reflog
```

Historial interno.

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)  
$ git reflog  
2314044 (HEAD -> master) HEAD@{0}: commit: Reorganiza archivos  
b12296a (feature/redes-sociales, feature/contacto) HEAD@{1}: checkout: moving from feature/redes-sociales to master  
b12296a (feature/redes-sociales, feature/contacto) HEAD@{2}: checkout: moving from master to feature/redes-sociales  
b12296a (feature/redes-sociales, feature/contacto) HEAD@{3}: checkout: moving fr  
on feature/contacto to master  
b12296a (feature/redes-sociales, feature/contacto) HEAD@{4}: checkout: moving from master to feature/contacto  
b12296a (feature/redes-sociales, feature/contacto) HEAD@{5}: commit: Agrega menú inicial  
fd94e4b HEAD@{6}: commit (initial): Primer commit
```

34) git blame

```
git blame README.md
```

Quién modificó cada línea.

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)
$ git blame README.md
Afd94e4b (Daniela Londoño 2026-05-27 15:27:27 -0500 1) Proyecto cafeteria cafe a
urora
```

PARTE 11 Tags y versiones

35) git tag

```
git tag v1.0
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)
$ git tag v1.0
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)
```

36) git tag -a

```
git tag -a v1.1 -m "Versión estable"
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)
$ git tag -a v1.1 -m "version estable"
```

37) git push --tags

```
git push --tags
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)
$ git push --tags
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 8 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (9/9), 935 bytes | 55.00 KiB/s, done.
Total 9 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Danielalon99/cafe-aurora.git
 * [new tag]          v1.0 -> v1.0
 * [new tag]          v1.1 -> v1.1
```

PARTE 12 Reset y revert

38) git reset

```
git reset HEAD README.md
```

Quitar del staging.

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)
$ git reset HEAD README.md
```

39) git revert

```
git revert HEAD
```

Deshacer commit sin borrar historial.

```
Revert "Reorganiza archivos"
This reverts commit 2314044f844d51fa831fae376490a29f79cb4f7a.
# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# On branch master
#
# Changes to be committed:
#   deleted:    menu-cafeteria.txt
#   renamed:    contacto.txt -> menu.txt
#
# Untracked files:
#   cafe-aurora/
#
dany.londono@hotmail.com (dany.londono@hotmail.com) está conectado
```

```
[master c66f0c7] Revert "Reorganiza archivos"
2 files changed, 0 insertions(+), 0 deletions(-)
delete mode 100644 menu-cafeteria.txt
rename contacto.txt => menu.txt (100%)
```

PARTE 13 Rebase

40) git rebase

```
git rebase main
```

Reorganiza historial.

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz (master)
$ git rebase master
Current branch master is up to date.
```

EXTRA Flujo profesional

```
git pull
git switch -c feature/menu
git add .
git commit -m "Actualiza menú"
git push origin feature/menu
```

RESUMEN VISUAL

Working Directory



git add



Staging Area



git commit



Repositorio local



git push



GitHub

LOS 40 COMANDOS APRENDIDOS

- 1 git init
- 2 git status
- 3 git config
- 4 git add
- 5 git commit
- 6 git log
- 7 git log --oneline
- 8 git diff
- 9 git branch
- 10 git checkout
- 11 git switch
- 12 git merge
- 13 git rm
- 14 git mv
- 15 git restore
- 16 git remote
- 17 git push
- 18 git pull
- 19 git fetch
- 20 git clone
- 21 git stash
- 22 git stash list
- 23 git stash pop
- 24 git show
- 25 git reflog
- 26 git blame

27 git tag
28 git tag -a
29 git push --tags
30 git reset
31 git revert
32 git rebase
33 git checkout -b
34 git remote add
35 git restore --staged
36 git branch feature/*
37 git switch main
38 git merge feature/*
39 git add .
40 git commit -m

¿Te atreves a realizar el siguiente reto?

Una **simulación real de empresa**, donde usaras:

- comandos básicos (add, commit, branch...)
- comandos intermedios (stash, rebase, reset...)
- comandos avanzados (reflog, bisect, cherry-pick...)
- inspección interna de Git
- herramientas de debugging
- configuración
- limpieza
- flujos de equipo

E-commerce Café Aurora pro

Trabaja en equipo con GitHub. Para desarrollar:

- tienda online
- pagos
- menú
- sistema de usuarios
- corrección de bugs

PARTE 1 CONFIGURACIÓN INICIAL (10 comandos)

```
git --version  
git config --global user.name "Juan"  
git config --global user.email juan@email.com
```

```
JSUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
$ git --version
git version 2.54.0.windows.1

JSUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
$ git config --global user.name "Daniela Londono"

JSUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
$ git config --global user.email "daniela.londono999@gmail.com"

JSUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
$ |
```

```
git config -list
```

```
MINGW64:/c/Users/JSUARIO/Aprendiz/CafeAuroraPro
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=schannel
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=master
user.name=Daniela Londono
user.email=daniela.londono999@gmail.com
core.repositoryformatversion=0
core.filemode=false
core.bare=false
core.logallrefupdates=true
core.symlinks=false
core.ignorecase=true
remote.origin.url=https://github.com/Danielalon99/cafe-aurora.git
remote.origin.fetch=+refs/heads/*:refs/remotes/origin/*
...skipping...
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=schannel
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=master
user.name=Daniela Londono
user.email=daniela.londono999@gmail.com
core.repositoryformatversion=0
core.filemode=false
core.bare=false
core.logallrefupdates=true
core.symlinks=false
core.ignorecase=true
remote.origin.url=https://github.com/Danielalon99/cafe-aurora.git
remote.origin.fetch=+refs/heads/*:refs/remotes/origin/*
~
```

```
git help
```

```

USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
$ git help
usage: git [-v | --version] [-h | --help] [-C <path>] [-c <name>=<value>]
[--exec-path=<path>] [--html-path] [--man-path] [--info-path]
[-p | --paginate | -P | --no-pager] [--no-replace-objects] [--no-lazy-fetch]
[--no-optional-locks] [--no-advice] [--bare] [--git-dir=<path>]
[--work-tree=<path>] [--namespace=<name>] [--config-env=<name>=<envvar>]
<command> [<args>]

These are common Git commands used in various situations:


start a working area (see also: git help tutorial)
  clone      Clone a repository into a new directory
  init       Create an empty Git repository or reinitialize an existing one


work on the current change (see also: git help everyday)
  add        Add file contents to the index
  mv         Move or rename a file, a directory, or a symlink
  restore    Restore working tree files
  rm         Remove files from the working tree and from the index


examine the history and state (see also: git help revisions)
  bisect     Use binary search to find the commit that introduced a bug
  diff       Show changes between commits, commit and working tree, etc
  grep       Print lines matching a pattern
  log        Show commit logs
  show       Show various types of objects
  status     Show the working tree status


grow, mark and tweak your common history
  backfill   Download missing objects in a partial clone
  branch     List, create, or delete branches
  commit     Record changes to the repository
  history     EXPERIMENTAL: Rewrite history
  merge      Join two or more development histories together
  rebase     Reapply commits on top of another base tip
  reset      Set 'HEAD' or the index to a known state
  switch     Switch branches
  tag        Create, list, delete or verify tags


collaborate (see also: git help workflows)
  fetch      Download objects and refs from another repository
  pull       Fetch from and integrate with another repository or a local branch
  push       Update remote refs along with associated objects

'git help -a' and 'git help -g' list available subcommands and some
concept guides. See 'git help <command>' or 'git help <concept>'
to read about a specific subcommand or concept.
See 'git help git' for an overview of the system.

```

git help commit



git-commit(1) Manual Page

NAME

git-commit - Record changes to the repository

SYNOPSIS

```

git commit [-a | --interactive | --patch] [-s] [-v] [-u[<mode>]] [--amend]
[--dry-run] [--c | -C | --squash] <commit> | --fixup [[amend] reword:]<commit>
[-F <file> | -m <msg>] [--reset-author] [--allow-empty]
[--allow-empty-message] [--no-verify] [-e] [--author=<author>]
[--date=<date>] [--cleanup=<mode>] [--[no-]status]
[-i | -o] [--pathspec-from-file=<file>] [--pathspec-file-null]
[(<--trailer <token>[=[:<value>]]...)] [-S[<keyid>]]
[<-->] [<pathspec>...]

```

git init

```

USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
$ git init
Initialized empty Git repository in C:/Users/USUARIO/Aprendiz/CafeAuroraPro/.git/

```

git clone <https://github.com/cafe/aurora.git>

```

USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
$ git clone https://github.com/Danielalon99/CAFE-AURORACAFE.git
Cloning into 'CAFE-AURORACAFE'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.

```

git remote -v

```

USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
$ git remote -v

```

git remote add origin URL

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
$ git remote add origin https://github.com/Danielalon99/CAFE-AURORACAFE
```

PARTE 2 INSPECCIÓN DEL PROYECTO (10 comandos)

git status

git status -s

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
  CAFE-AURORACAFE/

nothing added to commit but untracked files present (use "git add" to track)

USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
$ git status -s
?? CAFE-AURORACAFE/
```

git log

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
$ git log
commit 556b38f357c52a2aa9cd86f65af8188e53586ac8 (HEAD -> master)
Author: Daniela Londono <daniela.londono999@gmail.com>
Date: Mon Jun 1 22:52:03 2026 -0500

    pro cafe
```

git log -oneline

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
$ git log --oneline
556b38f (HEAD -> master) pro cafe
```

git log -graph

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
$ git log --graph
* commit 556b38f357c52a2aa9cd86f65af8188e53586ac8 (HEAD -> master)
  Author: Daniela Londono <daniela.londono999@gmail.com>
  Date: Mon Jun 1 22:52:03 2026 -0500

    pro cafe
```

git log -all

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
$ git log --all
commit 556b38f357c52a2aa9cd86f65af8188e53586ac8 (HEAD -> master)
Author: Daniela Londono <daniela.londono999@gmail.com>
Date: Mon Jun 1 22:52:03 2026 -0500

    pro cafe
```

git diff

git diff HEAD

git diff --staged

git show HEAD

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
$ git diff

USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
$ git diff HEAD

USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
$ git diff --staged

USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
$ git show HEAD
commit 556b38f357c52a2aa9cd86f65af8188e53586ac8 (HEAD -> master)
Author: Daniela Londono <daniela.londono999@gmail.com>
Date: Mon Jun 1 22:52:03 2026 -0500

    pro cafe

diff --git a/CAFE-AURORACAFE b/CAFE-AURORACAFE
new file mode 160000
index 0000000..4809bd6
--- /dev/null
+++ b/CAFE-AURORACAFE
@@ -0,0 +1 @@
-Subproject commit 4809bd64dd226406734e28132f90c0e94697b631
```


PARTE 3 WORKFLOW BÁSICO (10 comandos)

```
git add archivo.txt
git add .
```

```
JSUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
$ touch archivo.txt

JSUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
$ git add archivo.txt

JSUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
$ git add .
```

```
git add -A
git commit -m "mensaje"
git commit -am "mensaje"
```

```
JSUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
$ git add -A

JSUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
$ git commit -m "mensaje"
[master 30fa9fd] mensaje
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 archivo.txt

JSUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
$ git commit -am "mensaje"
On branch master
nothing to commit, working tree clean
```

```
git rm archivo.txt
```

```
JSUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
$ git rm archivo.txt
rm 'archivo.txt'
```

```
git mv viejo.txt nuevo.txt
```

```
JSUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
$ touch viejo.txt

JSUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
$ git add viejo.txt

JSUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
$ git commit -m "viejo"
[master c29e8bf] viejo
1 file changed, 0 insertions(+), 0 deletions(-)
rename archivo.txt => viejo.txt (100%)

JSUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
$ git mv viejo.txt nuevo.txt
```

```
git restore archivo.txt
```

```
git restore --staged archivo.txt
```

```
JSUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
$ git restore archivo.txt
```

```
JSUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
$ git restore --staged archivo.txt
```

```
git clean -f
```

```
JSUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
$ git clean -f
```

PARTE 4 RAMAS (12 comandos)

```
git branch
git branch feature/login
git branch -d feature/login
git branch -D feature/login
```

```
JSUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
git branch
master

JSUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
git branch feature/login

JSUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
git branch -d feature/login
deleted branch feature/login (was 25ac11e).

JSUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
git branch -D feature/login
error: branch 'feature/login' not found
```

```
git checkout main
```

```

USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
$ git checkout master
Already on 'master'

git checkout -b feature/login
git switch main
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
$ git checkout -b feature/login
Switched to a new branch 'feature/login'

USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (feature/login)
$ git switch master
Switched to branch 'master'

git switch -c feature/login
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
$ git switch -c feature/login
fatal: a branch named 'feature/login' already exists

git merge feature/login
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
$ git merge feature/login
Already up to date.

git merge --no-ff feature/login
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
$ git merge --no-ff feature/login
Already up to date.

git branch -m nuevo-nombre
git branch -r
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (master)
$ git branch -m nuevo-nombre

USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre)
$ git branch -r

```

PARTE 5 REBASE Y HISTORIAL (8 comandos)

```

git rebase main
git rebase -i HEAD~3
pick 30a91d # mensaje
pick c29e8bf # mensaje
pick 25a11e # archivo

# Rebase 556b38f..25a11e onto 556b38f (3 commands)
#
# Commands:
# p, pick <commit> = use commit
# r, reword <commit> = use commit, but edit the commit message
# e, edit <commit> = use commit, but stop for amending
# s, squash <commit> = use commit, but meld into previous commit
# f, fixup [-C | -c] <commit> = like "squash" but keep only the previous
#   commit's log message, unless -C is used, in which case
#   keep only this commit's message; -c is same as -C but
#   opens the editor
# x, exec <command> = run command (the rest of the line) using shell
# b, break = stop here (continue rebase later with 'git rebase --continue')
# d, drop <commit> = remove commit
# l, label <label> = label current HEAD with a name
# t, reset <label> = reset HEAD to a label
# m, merge [-C <commit>] [-c <commit>] <label> [# <oneline>]
#   create a merge commit using the original merge commit's
#   message (or the oneline, if no original merge commit was
#   specified); use -c <commit> to reword the commit message

git/rebase-merge/git-rebase-todo [unix] (23:32 01/06/2026) 12,1 Top
<Aprendiz/CafeAuroraPro/.git/rebase-merge/git-rebase-todo> [unix] 34L, 1548B

```

```

git rebase -abort
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre)
$ git rebase --abort
fatal: no rebase in progress

git rebase -continue
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre)
$ git rebase --continue
fatal: no rebase in progress

git cherry-pick abc1234

```

```
git revert HEAD
```

```
git reset HEAD~1
```

```
git reset --soft HEAD~1
```

PARTE 6 REMOTO (10 comandos)

```
git push
```

```
git push -u origin main
```

```
git push origin feature/login
```

```
git pull
```

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre)
$ git pull
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 873 bytes | 29.00 KiB/s, done.
From https://github.com/Danielalon99/CAFE-AURORACAFE
* [new branch]      main      -> origin/main
Already up to date.
```

```
git fetch
```

```
git fetch --all
```

```
git remote add origin URL
```

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre)
$ git fetch
```

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre)
$ git fetch --all
```

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre)
$ git remote add origin https://github.com/Danielalon99/CAFE-AURORACAFE
error: remote origin already exists.
```

```
git remote remove origin
```

```
git remote show origin
```

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre)
$ git remote remove origin
```

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre)
$ git remote show origin
fatal: 'origin' does not appear to be a git repository
fatal: Could not read from remote repository.
```

Please make sure you have the correct access rights
and the repository exists.

```
git ls-remote
```

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre)
$ git ls-remote origin
4809bd64dd226406734e28132f90c0e94697b631      HEAD
25ac11edd9690cbafce348ec67eb8080f064a0cb      refs/heads/feature/login
4809bd64dd226406734e28132f90c0e94697b631      refs/heads/main
4238abffe47afd1d7da4ac6d2493a30eb351a0ec      refs/heads/nuevo-nombre
```

PARTE 7 STASH (6 comandos)

```
git stash
```

```
git stash
warning: in the working copy of 'archivo.txt', LF will be replaced by CRLF the next time
  Git touches it
Saved working directory and index state WIP on nuevo-nombre: 4238abf Revert "archivo"
```

```
git stash list
```

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre)
$ git stash list
stash@{0}: WIP on nuevo-nombre: 4238abf Revert "archivo"
```

```
git stash pop
```

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre)
$ git stash pop
On branch nuevo-nombre
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   archivo.txt

Dropped refs/stash@{0} (00b5f90002931f48cf843f32bc41d1bf94ea93e1)
```

```
git stash apply
```

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre)
$ git stash apply
No stash entries found.
```

```
git stash drop
```

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre)
$ git stash drop
No stash entries found.
```

git stash clear

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre)
$ git stash clear
```

PARTE 8 RECUPERACIÓN Y DEBUG (10 comandos)

git reflog

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre)
$ git reflog
4238abf (HEAD -> nuevo-nombre) HEAD@{0}: reset: moving to HEAD
4238abf (HEAD -> nuevo-nombre) HEAD@{1}: revert: Revert "archivo"
25ac11e (feature/login) HEAD@{2}: rebase (finish): returning to refs/heads/nuevo-nombre
25ac11e (feature/login) HEAD@{3}: rebase (start): checkout HEAD~3
25ac11e (feature/login) HEAD@{4}: Branch: renamed refs/heads/master to refs/heads/nuevo-nombre
25ac11e (feature/login) HEAD@{6}: checkout: moving from feature/login to master
25ac11e (feature/login) HEAD@{7}: checkout: moving from master to feature/login
25ac11e (feature/login) HEAD@{8}: checkout: moving from master to master
25ac11e (feature/login) HEAD@{9}: commit: archivo
c29e8bf HEAD@{10}: commit: viejo
30fa9fd HEAD@{11}: commit: mensaje
556b38f HEAD@{12}: commit (initial): pro cafe
```

git reset --hard HEAD

git reset --mixed HEAD

git reset --soft HEAD

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre)
$ git reset --hard HEAD
HEAD is now at 4238abf Revert "archivo"
```

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre)
$ git reset --mixed HEAD
```

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre)
$ git reset --soft HEAD
```

git fsck

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre)
$ git fsck
Checking ref database: 100% (1/1), done.
Checking object directories: 100% (256/256), done.
dangling commit 00b5f90002931f48cf843f32bc41d1bf94ea93e1
dangling commit 4809bd64dd226406734e28132f90c0e94697b631
```

git gc

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre)
$ git gc
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 8 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (10/10), done.
Total 10 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
Enumerating cruft objects: 7, done.
Traversing cruft objects: 9, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (7/7), done.
Total 7 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
```

git blame archivo.txt

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre)
$ git status
git add archivo.txt
git commit -m "Recuperar archivo.txt"
On branch nuevo-nombre
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   archivo.txt

[nuevo-nombre ff43f7c] Recuperar archivo.txt
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 archivo.txt

USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre)
$ git blame archivo.txt
git show commit-id
```

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre)
$ git show c29e8bf
commit c29e8bfc20508f509f58bdb1ebd3f3d2c99255
Author: Daniela Londono <daniela.londono999@gmail.com>
Date:   Mon Jun 1 23:03:05 2026 -0500

    viejo

diff --git a/archivo.txt b/viejo.txt
similarity index 100%
rename from archivo.txt
rename to viejo.txt
```

```
git bisect start
```

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre)
$ git bisect start
status: waiting for both good and bad commits
```

```
git bisect bad
```

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre|BISECTING)
$ git bisect bad
status: waiting for good commit(s), bad commit known
```

```
git bisect Good
```

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre|BISECTING)
$ git bisect good
ff43f7c6c3b54c1510dec3f13161b3f5169cac4d was both good and bad
```

PARTE 9 VERSIONES (5 comandos)

```
git tag v1.0
```

```
USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre|BISECTING)
$ git tag v1.0
```

```
git tag -a v1.1 -m "release"
```

```
git tag
```

```
git push -tags
```

```

USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre|BISECTING)
$ git tag -a v1.1 -m "release"

USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre|BISECTING)
$ git tag
v1.0
v1.1

USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre|BISECTING)
$ git push --tags
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 432 bytes | 86.00 KiB/s, done.
Total 3 (delta 0), reused 1 (delta 0), pack-reused 0 (from 0)
To https://github.com/Danielalon99/CAFE-AURORACAFE.git
 * [new tag]          v1.0 -> v1.0
 * [new tag]          v1.1 -> v1.1

```

git show v1.0

```

USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre|BISECTING)
$ git show v1.0
commit ff43f7c6c3b54c1510dec3f13161b3f5169cac4d (HEAD -> nuevo-nombre, tag: v1.1, tag: v1.0)
Author: Daniela Londono <daniela.londono999@gmail.com>
Date:   Tue Jun 2 00:11:25 2026 -0500

    Recuperar archivo.txt

diff --git a/archivo.txt b/archivo.txt
new file mode 100644
index 0000000..e69de29

```

PARTE 10 LIMPIEZA (5 comandos)

```

git clean -n
git clean -f
git clean -fd
git prune

```

```

USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre|BISECTING)
$ git clean -n

USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre|BISECTING)
$ git clean -f
ggf

USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre|BISECTING)
$ git clean -fd

USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre|BISECTING)
$ git prune

```

git gc -aggressive

```

USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre)
$ git gc --aggressive
Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 8 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (12/12), done.
Total 12 (delta 3), reused 7 (delta 0), pack-reused 0 (from 0)
Enumerating cruft objects: 7, done.
Traversing cruft objects: 9, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (7/7), done.
Total 7 (delta 1), reused 7 (delta 1), pack-reused 0 (from 0)

```

PARTE 11 UTILIDADES AVANZADAS (5 comandos)

git shortlog


```

USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre)
$ git shortlog
Daniela Londono (6):
    pro cafe
    mensaje
    viejo
    archivo
    Revert "archivo"
    Recuperar archivo.txt

```

git archive

```

MINGW64/C:/Users/USUARIO/Aprendiz/CafeAuroraPro
$ git archive
usage: git archive [<options>] [<tree-ish>] [<path>...]
or: git archive --list
or: git archive --remote <repo> [--exec <cmd>] [<options>] [<tree-ish>] [<path>...]
or: git archive --remote <repo> [--exec <cmd>] --list

--[no-]format <fmt>    archive format
--[no-]prefix <prefix>  prepend prefix to each pathname in the archive
--[no-]add-file <file>  add untracked file to archive
--[no-]add-virtual-file <path:content>  add untracked file to archive
-o, --[no-]output <file>  write the archive to this file
--[no-]worktree-attributes  read .gitattributes in working directory
-v, --[no-]verbose      report archived files on stderr
--mtime <time>          set modification time of archive entries
-NUM                   set compression level
-l, --[no-]list         list supported archive formats
--[no-]remote <repo>    retrieve the archive from remote repository <repo>
--[no-]exec <command>    path to the remote git-unload-archive command

```

git describe

```

USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre)
$ git describe
1.1

```

git check-ignore

```

USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre)
$ git check-ignore -v viejo.txt

```

git update-index

```

USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre)
$ git update-index

```

PARTE 12 AYUDA Y CONTROL (5 comandos)

git help

```

USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre)
$ git help
usage: git [-v | --version] [-h | --help] [-C <path>] [-c <name>=<value>]
           [-e <exec-path>] [--html-path] [--man-path] [--info-path]
           [-p | --paginate] [-P | --no-pager] [--no-replace-objects] [--no-lazy-fetch]
           [--no-optional-locks] [--no-advice] [--bare] [--git-dir=<path>]
           [--work-tree=<path>] [--namespace=<name>] [--config-env=<name>=<envvar>]
           <command> [<args>]

These are common Git commands used in various situations:

start a working area (see also: git help tutorial)
  clone      Clone a repository into a new directory
  init       Create an empty Git repository or reinitialize an existing one

work on the current change (see also: git help everyday)
  add        Add file contents to the index
  mv         Move or rename a file, a directory, or a symlink
  restore    Restore working tree files
  rm         Remove files from the working tree and from the index

examine the history and state (see also: git help revisions)
  bisect     Use binary search to find the commit that introduced a bug
  diff       Show changes between commits, commit and working tree, etc
  grep       Print lines matching a pattern
  log        Show commit logs
  show       Show various types of objects
  status     Show the working tree status

grow, mark and tweak your common history
  backfill   Download missing objects in a partial clone
  branch     List, create, or delete branches
  commit     Record changes to the repository
  history     EXPERIMENTAL: Rewrite history
  merge      Join two or more development histories together
  rebase     Reapply commits on top of another base tip
  reset      Set 'HEAD' or the index to a known state
  switch     Switch branches
  tag        Create, list, delete or verify tags

collaborate (see also: git help workflows)
  fetch      Download objects and refs from another repository
  pull       Fetch from and integrate with another repository or a local branch
  push       Update remote refs along with associated objects

'git help -a' and 'git help -g' list available subcommands and some
concept guides. See 'git help <command>' or 'git help <concept>'
to read about a specific subcommand or concept.

```

git help merge

git help rebase

git help stash

git config --global -l


```

USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre)
$ git help merge

USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre)
$ git help rebase

USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre)
$ git help stash

USUARIO@DESKTOP-MH02JR7 MINGW64 ~/Aprendiz/CafeAuroraPro (nuevo-nombre)
$ git config --global -l
error: unknown switch 'l'
usage: git config list [<file-option>] [<display-option>] [--includes]
      or: git config get [<file-option>] [<display-option>] [--includes] [--all] [--regexp]
      [--value=<pattern>] [--fixed-value] [--default=<default>] [--url=<url>] <name>
      or: git config set [<file-option>] [--type=<type>] [--all] [--value=<pattern>] [--fixed-value] <name> <value>
      or: git config unset [<file-option>] [--all] [--value=<pattern>] [--fixed-value] <name>
      or: git config rename-section [<file-option>] <old-name> <new-name>
      or: git config remove-section [<file-option>] <name>
      or: git config edit [<file-option>]
      or: git config [<file-option>] --get-colorbool <name> [<stdout-is-tty>]

Config file location
  --[no-]global          use global config file
  --[no-]system          use system config file
  --[no-]local           use repository config file
  --[no-]worktree        use per-worktree config file
  -f, --[no-]file <file> use given config file
  --[no-]blob <blob-id> read config from given blob object

Action
  --get                  get value: name [<value-pattern>]
  --get-all             get all values: key [<value-pattern>]
  --get-regexp           get values for regexp: name-regex [<value-pattern>]
  --get-urlmatch         get value specific for the URL: section[.var] URL
  --replace-all         replace all matching variables: name value [<value-pattern>]
  --add                  add a new variable: name value
  --unset                remove a variable: name [<value-pattern>]
  --unset-all           remove all matches: name [<value-pattern>]
  --rename-section       rename section: old-name new-name
  --remove-section       remove a section: name
  -l, --list             list all
  -e, --edit             open an editor
  --get-color            find the color configured: slot [<default>]
  --get-colorbool        find the color setting: slot [<stdout-is-tty>]

Display options
  -z, --[no-]null        terminate values with NUL byte
  --[no-]name-only        show variable names only
  --[no-]show-origin      show origin of config (file, standard input, blob, command line)
  --[no-]show-scope       show scope of config (worktree, local, global, system, command)
  --[no-]show-names       show config keys in addition to their values

Type
  -t, --[no-]type <type> value is given this type
                        value is "true" or "false"
  --bool                  value is decimal number
  --int                   value is --bool or --int
  --bool-or-int           value is --bool or string
  --bool-or-str           value is a path (file or directory name)
  --path                  value is an expiry date
  --expiry-date           value is an expiry date

Other
  --[no-]default <value> with --get, use default value when missing entry
  --[no-]comment <value> human-readable comment string (# will be prepended as needed)
  --[no-]fixed-value      use string equality when comparing values to value pattern
  --[no-]includes         respect include directives on lookup

```

PARTE 13 FLUJO REAL DE TRABAJO (EQUIPO)

Ejemplo completo de trabajo profesional:

```
git pull
git checkout -b feature/payments
git add .
git commit -m "Add payments system"
git push origin feature/payments
git fetch
git rebase main
git merge feature/payments
git push
```

RESUMEN

En empresas se usa realmente esto:

20% de comandos hacen 80% del trabajo:

- git add
- git commit
- git push
- git pull
- git branch
- git checkout / switch
- git merge
- git stash
- git log
- git reset

IMPORTANTE

Muchos comandos avanzados como git bisect, git fsck, git gc
Se usan principalmente en:

- debugging
- mantenimiento
- Equipos o proyectos grandes
- recuperación avanzada

CONCLUSIÓN

Git no se aprende por cantidad de comandos, sino por:entender flujo:

Working Directory → Staging → Commit → Remote

RETO FINAL

Proyecto: E-commerce Café Aurora Pro

Trabajarás como si estuvieras en una empresa real usando:

- GitHub
- ramas por funcionalidades
- corrección de bugs
- trabajo en equipo
- rebase y merge
- debugging
- recuperación de errores
- versiones estables

Desafío Final

¿Serías capaz de:

- crear nuevas funcionalidades?
- resolver conflictos?
- recuperar commits eliminados?
- trabajar en equipo sin dañar el proyecto?
- desplegar versiones estables?

Meta del Taller

Al terminar este laboratorio podrás decir:

- “Sé usar Git en proyectos reales.”
- “Entiendo cómo trabajan los equipos de desarrollo.”
- “Puedo colaborar profesionalmente usando Git y GitHub.”

Objetivo del proyecto

Construir una tienda online para:

- vender café
- mostrar catálogo
- gestionar pedidos
- manejar usuarios
- integrar pagos
- publicar releases

ARQUITECTURA DEL REPOSITORIO

```
cafe-aurora/  
├── frontend/  
├── backend/  
├── docs/  
├── tests/  
├── .gitignore  
└── README.md
```

FASE 1 INICIO DEL PROYECTO

1. Clonar o iniciar repo

```
git clone https://github.com/cafe/aurora.git  
cd cafe-aurora
```

o si empiezas desde cero:

```
git init
```

2. Configuración del equipo

```
git config --global user.name "Developer 1"  
git config --global user.email "dev1@cafe.com"
```

3. Conectar remoto

```
git remote add origin https://github.com/cafe/aurora.git
```

FASE 2 PRIMER RELEASE BASE

4. Crear estructura inicial

```
git status
```

5. Agregar todo

```
git add .
```

6. Primer commit

```
git commit -m "chore: estructura inicial del proyecto"
```

7. Subir a GitHub

```
git push -u origin main
```

FASE 3 DESARROLLO POR RAMAS

8. Crear rama de desarrollo

```
git checkout -b develop
```

9. Crear feature catálogo

```
git checkout -b feature/catalogo
```

10. Trabajar catálogo

Se agregan productos:

- café colombiano
- café espresso
- café premium

11. Guardar cambios

```
git add .  
git commit -m "feat: catálogo de productos inicial"
```

12. Subir rama

```
git push origin feature/catalogo
```

13. Merge a develop

```
git checkout develop  
git merge feature/catalogo
```

FASE 4 SISTEMA DE PAGOS

14. Crear rama

```
git checkout -b feature/pagos
```

15. Cambios en pagos

Se agregan:

- PayPal
- tarjeta
- Stripe

16. Guardar cambios

```
git add .  
git commit -m "feat: integración de pagos"
```

17. Subir rama

```
git push origin feature/pagos
```

FASE 5 CONFLICTO REAL (IMPORTANTE)

Dos personas modifican el mismo archivo:

```
checkout.html
```

18. Merge conflict

```
git checkout develop  
git merge feature/pagos
```

Conflicto aparece.

19. Ver estado

```
git status
```

20. Resolver conflicto manualmente

Editas archivo y decides versión correcta.

21. Marcar como resuelto

```
git add checkout.html  
git commit -m "fix: resolve merge conflict en pagos"
```

FASE 6 RELEASE

22. Crear rama release

```
git checkout -b release/v1.0
```

23. Ajustes finales

- bugs menores
- UI
- performance

24. Commit final

```
git commit -am "release: versión 1.0 lista"
```

25. Merge a main

```
git checkout main  
git merge release/v1.0
```

26. Tag de versión

```
git tag -a v1.0 -m "Primera versión estable"
```

27. Subir todo

```
git push origin main  
git push --tags
```

FASE 7 TRABAJO DE EMERGENCIA (HOTFIX)

28. Bug crítico en pagos

```
git checkout -b hotfix/pagos-error
```

29. Fix rápido

```
git commit -am "hotfix: error crítico en checkout"
```

30. Merge inmediato

```
git checkout main  
git merge hotfix/pagos-error
```

FASE 8 HISTORIAL Y CONTROL

31. Ver historial

```
git log --oneline --graph
```

32. Ver cambios de un commit

```
git show HEAD
```

33. Quién modificó qué

```
git blame backend/payments.js
```

34. Recuperar estado anterior

```
git reflog
```

FASE 9 STASH (trabajo urgente)

35. Guardar cambios temporales

```
git stash
```

36. Volver cambios

```
git stash pop
```


FASE 10 REBASE (historial limpio)

37. Reorganizar commits

```
git rebase main
```

38. Rebase interactivo

```
git rebase -i HEAD~3
```

FASE 11 DEBUG AVANZADO

39. Buscar bug

```
git bisect start
git bisect bad
git bisect good v0.9
```

40. Limpiar repo

```
git gc
git fsck
```

FLUJO FINAL DEL PROYECTO

```
feature → develop → release → main → tag
    ↓           ↓           ↓           ↓
    testing    staging    producción  versión
```

LO QUE APRENDISTE REALIZANDO EL ANTERIOR PROYECTO

Flujo real de empresa

- feature branches
- develop branch
- release branch

- hotfix branch

Trabajo en equipo

- merges
- conflictos
- rebase

Producción real

- tags
- versiones
- hotfix

Control total

- logs
- reflog
- blame
- bisect